

Adapter Pattern in JavaScript

What is the Adapter Pattern?

The **Adapter Pattern** is a structural design pattern that allows **objects with incompatible interfaces to work together**.

In simple terms:

"Convert one interface into another interface that the client expects."

It acts like a real-world adapter:

Example:

A UK power plug:

```
graph TD; UKPlug[UK Plug] --- Adapter[Adapter]; Adapter --- EuropeanSocket[European Socket];
```

The adapter does not change the plug or socket. It simply allows them to communicate.

Why Does the Adapter Pattern Exist?

A common problem in software development is integrating systems that were not designed to work together.

Example:

Your application expects:

```
id="expected"  
payment.process(amount);
```

But a third-party payment service provides:

```
id="thirdparty"  
stripe.makePayment(value);
```

The interfaces are different:

Your application:

```
process()
```

External library:

```
makePayment()
```

They cannot communicate directly.

The Adapter solves this:

```
id="adapter"  
Application  
  
  |  
  v  
  
Payment Adapter  
  
  |  
  v  
  
Stripe API
```

Basic Adapter Pattern Example

Payment System Example

Imagine an application that already supports:

```
id="oldSystem"  
payment.pay();
```

Your new payment provider uses:

```
id="newSystem"  
stripe.charge();
```

You cannot modify Stripe's code.

The Adapter translates between them.

Existing Application Interface

The application expects:

```
id="paymentInterface"  
class PaymentProcessor {  
  
    pay(amount){  
  
        console.log(  
            `Paid ${amount}`  
        );  
    }  
  
}
```

The application knows:

```
id="client"  
processor.pay();
```

Third-Party Service

Imagine this is a library you cannot modify.

```
id="stripe"  
class StripePayment {  
  
    charge(amount){  
  
        console.log(  
            `Stripe charged ${amount}`  
        );  
    }  
  
}
```

```
    );  
  
    }  
  
}
```

The problem:

The methods have different names.

Application:

```
pay()
```

Stripe:

```
charge()
```

Creating the Adapter

The Adapter translates the interface.

```
id="adapterClass"  
class StripeAdapter {  
  
    constructor(){  
  
        this.stripe =  
            new StripePayment();  
  
    }  
  
    pay(amount){  
  
        // Convert the expected  
        // pay() method into Stripe's  
        // charge() method.  
  
        this.stripe.charge(amount);  
  
    }  
  
}
```

```
}
```

Using the Adapter

The application does not know Stripe exists.

```
id="usingAdapter"  
const payment =  
    new StripeAdapter();  
  
payment.pay(100);
```

Output:

```
Stripe charged 100
```

The application thinks it is using:

```
pay()
```

but internally it uses:

```
charge()
```

How This Works Step-by-Step

1. Client Expects an Interface

The application expects:

```
id="step1"  
payment.pay(amount);
```

2. Existing System Has Different Interface

Stripe provides:

```
id="step2"  
stripe.charge(amount);
```

The two systems cannot communicate.

3. Adapter Translates

The Adapter receives:

```
id="step3"  
pay(100)
```

and converts it:

```
id="step4"  
charge(100)
```

4. Third-Party System Executes

Stripe processes:

```
id="step5"  
charge(100);
```

The application never needs to know the difference.

Another Example: Weather API Adapter

Imagine your application expects:

```
id="weatherExpected"  
weather.getTemperature();
```

But an external API provides:

```
id="weatherAPI"  
weatherAPI.fetchTemp();
```

Third-Party API

```
id="weatherClass"  
class ExternalWeatherAPI {  
  
    fetchTemp(){  
  
        return 25;  
  
    }  
  
}
```

Adapter

```
id="weatherAdapter"  
class WeatherAdapter {  
  
    constructor(){  
  
        this.api =  
            new ExternalWeatherAPI();  
  
    }  
  
    getTemperature(){  
  
        return this.api.fetchTemp();  
  
    }  
  
}
```

Application

```
id="weatherUse"  
const weather =  
  new WeatherAdapter();  
  
console.log(  
  weather.getTemperature()  
);
```

Output:

```
25
```

The application uses:

```
getTemperature()
```

while the API uses:

```
fetchTemp()
```

A More Realistic JavaScript Example

Logging System Adapter

Imagine your application uses:

```
id="loggerExpected"  
logger.log(message);
```

But a third-party logging library uses:

```
id="externalLogger"  
externalLogger.writeLog(message);
```

External Logger

```
id="externalLoggerCode"
class ThirdPartyLogger {

    writeLog(message){

        console.log(
            `[LOG]: ${message}`
        );

    }

}
```

Adapter

```
id="loggerAdapter"
class LoggerAdapter {

    constructor(){

        this.logger =
            new ThirdPartyLogger();

    }

    log(message){

        this.logger.writeLog(message);

    }

}
```

Application

```
id="loggerApplication"
const logger =
    new LoggerAdapter();
```

```
logger.log(  
    "User logged in"  
);
```

Output:

```
[LOG]: User logged in
```

Common Uses of Adapter Pattern in JavaScript

1. Third-Party API Integration

Example:

```
id="apiAdapter"  
PaymentAdapter.process();
```

Used with:

- Payment providers
- Maps APIs
- Authentication providers
- Cloud services

Problem solved:

Allow external services to work with your existing application.

2. Legacy Code Integration

Example:

Old system:

```
id="legacy"  
oldDatabase.getUserData();
```

New application:

```
id="new"
userRepository.findUser();
```

Adapter:

```
id="legacyAdapter"
findUser(){

    return oldDatabase.getUserData();

}
```

Problem solved:

Use old code without rewriting it.

3. Different Data Formats

Example:

Your application expects:

```
id="json"
{
  name: "John"
}
```

External API returns:

```
id="xml"
<User>
<Name>John</Name>
</User>
```

Adapter converts:

```
id="convert"
XML → JSON
```

Problem solved:

Make incompatible data structures compatible.

4. Database Switching

Example:

Application expects:

```
id="databaseInterface"  
database.findUsers();
```

Database A:

```
id="mongo"  
mongo.queryUsers();
```

Database B:

```
id="sql"  
sql.selectUsers();
```

Adapters allow switching databases without changing application code.

5. Frontend Libraries

Example:

Your application expects:

```
id="frontendExpected"  
button.render();
```

A UI library provides:

```
id="frontendLibrary"  
component.mount();
```

Adapter connects them.

Problems Adapter Pattern Solves

Problem 1: Incompatible Interfaces

Without Adapter:

```
id="incompatible"  
Application  
  
expects:  
pay()  
  
Provider:  
  
charge()
```

They cannot work together.

With Adapter:

```
id="compatible"  
Application  
  
pay()  
  
|  
  
Adapter  
  
|  
  
charge()
```

Problem 2: Avoid Modifying Existing Code

Without Adapter:

```
id="modify"  
Change third-party library
```

Problems:

- May break updates.

- May introduce bugs.
- May not be possible.

With Adapter:

```
id="wrapper"  
Create a wrapper around it.
```

Problem 3: Reduce Coupling

Without Adapter:

```
id="tight"  
Application  
  |  
  v  
Specific Library
```

The application depends directly on the library.

With Adapter:

```
id="loose"  
Application  
  
  |  
Adapter  
  
  |  
Library
```

The application only depends on the adapter.

Adapter vs Facade Pattern

Adapter	Facade
Makes incompatible systems work together	Simplifies a complex system

Adapter	Facade
Changes an interface	Creates a simpler interface
Usually wraps one object	Usually wraps many objects
Focuses on compatibility	Focuses on usability

Example Adapter:

```
id="adapterExample"
OldPaymentAdapter.pay();
```

Makes old payment system work.

Example Facade:

```
id="facadeExample"
VideoEditor.export();
```

Hides multiple video editing steps.

Adapter vs Decorator Pattern

Adapter	Decorator
Changes interface	Adds behaviour
Makes objects compatible	Extends functionality
Usually changes method names/data	Keeps same interface

Adapter:

```
id="adapterDecorator"
oldAPI.request()
```

becomes:

```
newAPI.fetch()
```

Decorator:

```
id="decorator"  
coffee.addMilk();
```

adds functionality.

Adapter vs Factory Pattern

Adapter	Factory
Converts existing objects	Creates new objects
Structural pattern	Creational pattern
Focuses on compatibility	Focuses on object creation

Adapter:

```
id="adapterFactory"  
StripeAdapter
```

Factory:

```
id="factory"  
PaymentFactory.create("stripe");
```

Types of Adapter Pattern

1. Object Adapter

Uses composition.

Example:

```
id="objectAdapter"  
class Adapter {  
  
    constructor(){  
  
        this.service =  
            new Service();  
  
    }  
}
```

```
}
```

Most common in JavaScript.

2. Class Adapter

Uses inheritance.

Example:

```
id="classAdapter"  
class Adapter extends OldService {  
  
}
```

Less common in JavaScript because JavaScript uses prototypal inheritance and composition is usually preferred.

Disadvantages of Adapter Pattern

1. Extra Code

You create another layer:

```
Application  
  |  
  Adapter  
  |  
  Service
```

For simple cases, this may be unnecessary.

2. Can Hide Problems

An adapter can sometimes hide a poorly designed interface instead of fixing it.

3. Too Many Adapters

Large applications may end up with many adapter classes.

When Should You Use Adapter?

Use Adapter when:

- ✓ You need to integrate third-party libraries
- ✓ Existing interfaces do not match
- ✓ You need to support legacy systems
- ✓ You want to isolate external dependencies

Examples:

- Payment providers
- APIs
- Database systems
- Authentication providers
- File formats

Avoid Adapter when:

- ✗ You control both systems and can redesign them
- ✗ The conversion logic is trivial
- ✗ A simpler solution exists

Key Takeaway

The Adapter Pattern separates:

```
graph TD
    A[Incompatible Interface] --> B[Adapter]
    B --> C[Compatible Interface]
```

Instead of:

```
graph TD
    A[Application] --> B[Cannot communicate with external system]
```

In JavaScript, the Adapter Pattern is commonly implemented by:

1. Creating a wrapper class or function.

2. Translating method names or data formats.
3. Keeping third-party or legacy code isolated.

It is mainly used to **make existing or external code work with your application without modifying the original code.**